

Module 7: Plotting and Visualization (Matplotlib and Seaborn)

Author: Lei Wu

Date: 2023-01-01

Version: 1.0.0

Learning Objectives

By the end of this module, you will be able to:

- **Master Matplotlib fundamentals:** Create, customize, and style various plot types
- **Apply statistical visualization:** Use Seaborn for advanced statistical graphics
- **Design publication-quality plots:** Implement professional styling and layout techniques
- **Conduct exploratory data analysis:** Use visualization to discover patterns and insights
- **Export and share visualizations:** Save plots in multiple formats for different purposes

Module Overview

This module provides comprehensive coverage of data visualization in Python, progressing from basic plotting concepts to advanced statistical graphics. You'll learn to create publication-quality visualizations that effectively communicate data insights.

Topics Covered

1. **Introduction to Matplotlib** - Foundation and core concepts
2. **Basic Plotting Functions** - Essential plot types and syntax
3. **Figure and Axes Management** - Object-oriented plotting approach
4. **Plot Customization and Styling** - Professional appearance and branding
5. **Different Plot Types** - Line, bar, scatter, histogram, and more
6. **Introduction to Seaborn** - Statistical visualization library
7. **Statistical Plots and Distributions** - Advanced statistical graphics
8. **Categorical Plots** - Visualizing categorical data effectively
9. **Matrix Plots and Heatmaps** - Correlation and pattern visualization
10. **Subplots and Multiple Plots** - Complex layouts and multi-panel figures
11. **Saving and Exporting Plots** - Production-ready output formats

Prerequisites

- Basic Python programming (Modules 1-2)
- NumPy fundamentals (Module 3)
- Pandas basics (Modules 4-6)

Key Libraries

- **Matplotlib:** Core plotting library with fine-grained control
- **Seaborn:** High-level statistical visualization
- **NumPy:** Numerical data generation and manipulation
- **Pandas:** Data handling and integration

1. Introduction to Matplotlib

What is Matplotlib?

Matplotlib is the foundational plotting library in Python, providing a MATLAB-like interface for creating static, animated, and interactive visualizations. It's the backbone of Python's data visualization ecosystem.

Key Features

- **Multi-platform:** Works on Windows, macOS, and Linux
- **Multiple backends:** Supports various output formats (PNG, PDF, SVG, etc.)
- **Object-oriented and procedural interfaces:** Flexible programming approaches
- **Integration:** Works seamlessly with NumPy, Pandas, and Jupyter notebooks
- **Customization:** Fine-grained control over every aspect of plots

Matplotlib Architecture

Matplotlib follows a hierarchical structure:

- **Figure:** The top-level container for all plot elements
- **Axes:** The actual plotting area where data is visualized
- **Axis:** The numerical axis (x-axis, y-axis)
- **Artist:** Individual elements like lines, text, patches

```
In [ ]: # Import necessary libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
from matplotlib.patches import Rectangle, Circle
import warnings
warnings.filterwarnings('ignore')

# Set up the plotting style
plt.style.use('default')
```

```

sns.set_palette("husl")

# 1. Introduction to Matplotlib - Basic Setup
print("=== INTRODUCTION TO MATPLOTLIB ===")

# Basic import and setup
print("Matplotlib version:", plt.matplotlib.__version__)

# Create a simple plot to demonstrate basic functionality
x = np.linspace(0, 10, 100)
y = np.sin(x)

# Method 1: Pyplot interface (procedural)
plt.figure(figsize=(8, 6))
plt.plot(x, y, label='sin(x)', linewidth=2)
plt.title('Basic Matplotlib Plot')
plt.xlabel('x')
plt.ylabel('y')
plt.legend()
plt.grid(True, alpha=0.3)
plt.show()

# Method 2: Object-oriented interface
fig, ax = plt.subplots(figsize=(8, 6))
ax.plot(x, y, label='sin(x)', linewidth=2)
ax.set_title('Object-Oriented Matplotlib Plot')
ax.set_xlabel('x')
ax.set_ylabel('y')
ax.legend()
ax.grid(True, alpha=0.3)
plt.show()

# Understanding the matplotlib hierarchy
print("\nMatplotlib Hierarchy:")
print("Figure -> Axes -> Axis -> Artist")
print("- Figure: Top-level container")
print("- Axes: Plotting area")
print("- Axis: X/Y axis")
print("- Artist: Individual elements")

# Demonstrate figure and axes properties
fig, ax = plt.subplots(figsize=(10, 6))

# Add some data
x = np.linspace(0, 2*np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# Plot multiple lines
ax.plot(x, y1, label='sin(x)', linewidth=2, color='blue')
ax.plot(x, y2, label='cos(x)', linewidth=2, color='red')

# Customize the plot
ax.set_title('Understanding Matplotlib Components', fontsize=16, fontweight='bold')
ax.set_xlabel('Angle (radians)', fontsize=12)
ax.set_ylabel('Value', fontsize=12)
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3)

# Add annotations

```

```
ax.annotate('Maximum', xy=(np.pi/2, 1), xytext=(np.pi/2 + 0.5, 1.2),
            arrowprops=dict(arrowstyle='->', color='black'),
            fontsize=10, ha='center')

plt.tight_layout()
plt.show()

print("\n=== END OF MATPLOTLIB INTRODUCTION ===")
```

2. Basic Plotting Functions

Essential Plot Types

Matplotlib provides a comprehensive set of plotting functions for different data types and visualization needs:

Line Plots (`plt.plot()`)

- **Purpose:** Show trends and relationships over continuous data
- **Use cases:** Time series, mathematical functions, continuous relationships
- **Key parameters:** `x`, `y`, `color`, `linewidth`, `linestyle`, `marker`

Scatter Plots (`plt.scatter()`)

- **Purpose:** Show relationships between two variables
- **Use cases:** Correlation analysis, clustering, outlier detection
- **Key parameters:** `x`, `y`, `c` (color), `s` (size), `alpha` (transparency)

Bar Plots (`plt.bar()`)

- **Purpose:** Compare categorical data
- **Use cases:** Counts, frequencies, comparisons
- **Key parameters:** `x`, `height`, `width`, `color`, `edgecolor`

Histograms (`plt.hist()`)

- **Purpose:** Show distribution of numerical data
- **Use cases:** Data exploration, distribution analysis
- **Key parameters:** `x`, `bins`, `alpha`, `density`, `cumulative`

Pie Charts (`plt.pie()`)

- **Purpose:** Show proportions of a whole
- **Use cases:** Composition analysis, market share
- **Key parameters:** `x`, `labels`, `autopct`, `explode`, `colors`

```
In [ ]: # 2. Basic Plotting Functions – Comprehensive Examples
print("=== BASIC PLOTTING FUNCTIONS ===")
```

```

# Generate sample data
np.random.seed(42)
x = np.linspace(0, 10, 50)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.sin(x) * np.cos(x)

# Create a comprehensive example showing all basic plot types
fig, axes = plt.subplots(2, 3, figsize=(15, 10))
fig.suptitle('Basic Matplotlib Plot Types', fontsize=16, fontweight='bold')

# 1. Line Plot
axes[0, 0].plot(x, y1, 'b-', linewidth=2, label='sin(x)')
axes[0, 0].plot(x, y2, 'r--', linewidth=2, label='cos(x)')
axes[0, 0].set_title('Line Plot')
axes[0, 0].set_xlabel('x')
axes[0, 0].set_ylabel('y')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# 2. Scatter Plot
x_scatter = np.random.randn(100)
y_scatter = 2 * x_scatter + np.random.randn(100) * 0.5
colors = np.random.rand(100)
axes[0, 1].scatter(x_scatter, y_scatter, c=colors, alpha=0.6, s=50)
axes[0, 1].set_title('Scatter Plot')
axes[0, 1].set_xlabel('X')
axes[0, 1].set_ylabel('Y')

# 3. Bar Plot
categories = ['A', 'B', 'C', 'D', 'E']
values = [23, 45, 56, 78, 32]
colors_bar = ['red', 'green', 'blue', 'orange', 'purple']
axes[0, 2].bar(categories, values, color=colors_bar, edgecolor='black', l
axes[0, 2].set_title('Bar Plot')
axes[0, 2].set_xlabel('Categories')
axes[0, 2].set_ylabel('Values')

# 4. Histogram
data_hist = np.random.normal(0, 1, 1000)
axes[1, 0].hist(data_hist, bins=30, alpha=0.7, color='skyblue', edgecolor
axes[1, 0].set_title('Histogram')
axes[1, 0].set_xlabel('Value')
axes[1, 0].set_ylabel('Frequency')

# 5. Pie Chart
sizes = [30, 25, 20, 15, 10]
labels = ['Category 1', 'Category 2', 'Category 3', 'Category 4', 'Catego
colors_pie = ['gold', 'lightcoral', 'lightskyblue', 'lightgreen', 'plum']
explode = (0.1, 0, 0, 0, 0) # explode the first slice
axes[1, 1].pie(sizes, labels=labels, colors=colors_pie, autopct='%1.1f%%'
                explode=explode, shadow=True, startangle=90)
axes[1, 1].set_title('Pie Chart')

# 6. Box Plot
data_box = [np.random.normal(0, 1, 100),
            np.random.normal(1, 1.5, 100),
            np.random.normal(2, 0.5, 100)]
axes[1, 2].boxplot(data_box, labels=['Group 1', 'Group 2', 'Group 3'])

```

```

axes[1, 2].set_title('Box Plot')
axes[1, 2].set_ylabel('Value')

plt.tight_layout()
plt.show()

# Advanced Line Plot Examples
print("\nAdvanced Line Plot Examples:")

# Create a more sophisticated line plot
fig, ax = plt.subplots(figsize=(12, 8))

# Multiple lines with different styles
x = np.linspace(0, 4*np.pi, 200)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.sin(x + np.pi/4)
y4 = np.cos(x + np.pi/4)

# Plot with different line styles and markers
ax.plot(x, y1, 'b-', linewidth=2, label='sin(x)', alpha=0.8)
ax.plot(x, y2, 'r--', linewidth=2, label='cos(x)', alpha=0.8)
ax.plot(x, y3, 'g:', linewidth=3, label='sin(x +  $\pi/4$ )', alpha=0.8)
ax.plot(x, y4, 'm-.', linewidth=2, label='cos(x +  $\pi/4$ )', alpha=0.8)

# Add markers at specific points
marker_x = np.linspace(0, 4*np.pi, 20)
ax.plot(marker_x, np.sin(marker_x), 'ko', markersize=6, alpha=0.6, label=

# Customize the plot
ax.set_title('Advanced Line Plot with Multiple Styles', fontsize=16, font
ax.set_xlabel('Angle (radians)', fontsize=12)
ax.set_ylabel('Value', fontsize=12)
ax.legend(fontsize=10, loc='upper right')
ax.grid(True, alpha=0.3)
ax.set_xlim(0, 4*np.pi)

# Add horizontal and vertical lines
ax.axhline(y=0, color='black', linestyle='-', alpha=0.3)
ax.axvline(x=2*np.pi, color='red', linestyle='--', alpha=0.5, label='x =

plt.tight_layout()
plt.show()

# Scatter Plot with Color Mapping
print("\nAdvanced Scatter Plot with Color Mapping:")

# Generate data for scatter plot
n = 100
x_scatter = np.random.randn(n)
y_scatter = 2 * x_scatter + np.random.randn(n) * 0.5
colors_scatter = np.random.rand(n)
sizes_scatter = 1000 * np.random.rand(n)

fig, ax = plt.subplots(figsize=(10, 6))
scatter = ax.scatter(x_scatter, y_scatter, c=colors_scatter, s=sizes_scatter,
                    alpha=0.6, cmap='viridis')
ax.set_title('Advanced Scatter Plot with Color and Size Mapping')
ax.set_xlabel('X')
ax.set_ylabel('Y')

```

```
plt.colorbar(scatter, label='Color Value')
plt.show()

print("\n=== END OF BASIC PLOTTING FUNCTIONS ===")
```

3. Figure and Axes Management

Object-Oriented Approach

The object-oriented approach provides more control and is recommended for complex plots:

Figure Management

- **plt.figure()** : Create a new figure
- **fig.add_subplot()** : Add subplots to a figure
- **plt.subplots()** : Create figure and axes simultaneously
- **fig.savefig()** : Save figure to file

Axes Management

- **ax.plot()** : Plot on specific axes
- **ax.set_xlabel()** : Set x-axis label
- **ax.set_ylabel()** : Set y-axis label
- **ax.set_title()** : Set plot title
- **ax.legend()** : Add legend
- **ax.grid()** : Add grid

Subplot Layouts

- **plt.subplot(nrows, ncols, index)** : Create subplots
- **plt.subplots(nrows, ncols)** : Create multiple subplots
- **plt.subplot2grid()** : Create subplots with custom grid
- **plt.GridSpec()** : Advanced subplot layouts

```
In [ ]: # 3. Figure and Axes Management – Object-Oriented Approach
print("=== FIGURE AND AXES MANAGEMENT ===")

# Method 1: Using plt.subplots() – Most Common
print("Method 1: Using plt.subplots()")
fig, axes = plt.subplots(2, 2, figsize=(12, 10))
fig.suptitle('Figure and Axes Management – Method 1', fontsize=16, fontwe

# Generate data
x = np.linspace(0, 2*np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.tan(x)
y4 = np.exp(-x/5) * np.sin(x)

# Plot on different axes
```

```

axes[0, 0].plot(x, y1, 'b-', linewidth=2)
axes[0, 0].set_title('sin(x)')
axes[0, 0].set_xlabel('x')
axes[0, 0].set_ylabel('y')
axes[0, 0].grid(True, alpha=0.3)

axes[0, 1].plot(x, y2, 'r-', linewidth=2)
axes[0, 1].set_title('cos(x)')
axes[0, 1].set_xlabel('x')
axes[0, 1].set_ylabel('y')
axes[0, 1].grid(True, alpha=0.3)

axes[1, 0].plot(x, y3, 'g-', linewidth=2)
axes[1, 0].set_title('tan(x)')
axes[1, 0].set_xlabel('x')
axes[1, 0].set_ylabel('y')
axes[1, 0].grid(True, alpha=0.3)
axes[1, 0].set_ylim(-5, 5) # Limit y-axis for tan function

axes[1, 1].plot(x, y4, 'm-', linewidth=2)
axes[1, 1].set_title('exp(-x/5) * sin(x)')
axes[1, 1].set_xlabel('x')
axes[1, 1].set_ylabel('y')
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Method 2: Using plt.subplot() - Individual subplots
print("\nMethod 2: Using plt.subplot()")
plt.figure(figsize=(12, 8))

# First subplot
plt.subplot(2, 3, 1)
plt.plot(x, y1, 'b-', linewidth=2)
plt.title('sin(x)')
plt.grid(True, alpha=0.3)

# Second subplot
plt.subplot(2, 3, 2)
plt.plot(x, y2, 'r-', linewidth=2)
plt.title('cos(x)')
plt.grid(True, alpha=0.3)

# Third subplot
plt.subplot(2, 3, 3)
plt.plot(x, y3, 'g-', linewidth=2)
plt.title('tan(x)')
plt.grid(True, alpha=0.3)
plt.ylim(-5, 5)

# Fourth subplot (spans 2 columns)
plt.subplot(2, 3, (4, 5))
plt.plot(x, y4, 'm-', linewidth=2)
plt.title('exp(-x/5) * sin(x)')
plt.grid(True, alpha=0.3)

# Sixth subplot
plt.subplot(2, 3, 6)
plt.hist(np.random.normal(0, 1, 1000), bins=30, alpha=0.7, color='orange')

```



```

plt.title('Normal Distribution')
plt.grid(True, alpha=0.3)

plt.suptitle('Figure and Axes Management – Method 2', fontsize=16, fontwe
plt.tight_layout()
plt.show()

# Method 3: Using subplot2grid for complex layouts
print("\nMethod 3: Using subplot2grid for complex layouts")
fig = plt.figure(figsize=(15, 10))

# Main plot (large)
ax1 = plt.subplot2grid((3, 3), (0, 0), colspan=2, rowspan=2)
ax1.plot(x, y1, 'b-', linewidth=2, label='sin(x)')
ax1.plot(x, y2, 'r-', linewidth=2, label='cos(x)')
ax1.set_title('Main Plot – Trigonometric Functions', fontsize=14, fontwei
ax1.set_xlabel('x')
ax1.set_ylabel('y')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Small plots
ax2 = plt.subplot2grid((3, 3), (0, 2))
ax2.hist(np.random.normal(0, 1, 500), bins=20, alpha=0.7, color='skyblue'
ax2.set_title('Distribution')

ax3 = plt.subplot2grid((3, 3), (1, 2))
ax3.bar(['A', 'B', 'C', 'D'], [23, 45, 56, 78], color=['red', 'green', 'b
ax3.set_title('Bar Chart')

# Bottom plot (spans full width)
ax4 = plt.subplot2grid((3, 3), (2, 0), colspan=3)
ax4.plot(x, y4, 'm-', linewidth=2)
ax4.fill_between(x, y4, alpha=0.3, color='magenta')
ax4.set_title('Damped Oscillation', fontsize=12)
ax4.set_xlabel('x')
ax4.set_ylabel('y')
ax4.grid(True, alpha=0.3)

plt.suptitle('Complex Layout with subplot2grid', fontsize=16, fontweight=
plt.tight_layout()
plt.show()

# Method 4: Using GridSpec for advanced layouts
print("\nMethod 4: Using GridSpec for advanced layouts")
from matplotlib.gridspec import GridSpec

fig = plt.figure(figsize=(15, 10))
gs = GridSpec(3, 3, figure=fig)

# Create axes with GridSpec
ax1 = fig.add_subplot(gs[0, :]) # Top row, all columns
ax2 = fig.add_subplot(gs[1, 0]) # Middle row, first column
ax3 = fig.add_subplot(gs[1, 1]) # Middle row, second column
ax4 = fig.add_subplot(gs[1, 2]) # Middle row, third column
ax5 = fig.add_subplot(gs[2, :]) # Bottom row, all columns

# Plot on each axis
ax1.plot(x, y1, 'b-', linewidth=2, label='sin(x)')
ax1.plot(x, y2, 'r-', linewidth=2, label='cos(x)')

```

```

ax1.set_title('Top Plot - Trigonometric Functions')
ax1.legend()
ax1.grid(True, alpha=0.3)

ax2.hist(np.random.normal(0, 1, 500), bins=20, alpha=0.7, color='skyblue')
ax2.set_title('Histogram')

ax3.bar(['A', 'B', 'C'], [23, 45, 56], color=['red', 'green', 'blue'])
ax3.set_title('Bar Chart')

ax4.pie([30, 25, 20, 15, 10], labels=['A', 'B', 'C', 'D', 'E'], autopct='
ax4.set_title('Pie Chart')

ax5.scatter(np.random.randn(100), np.random.randn(100), alpha=0.6, c=np.r
ax5.set_title('Scatter Plot')
ax5.grid(True, alpha=0.3)

plt.suptitle('Advanced Layout with GridSpec', fontsize=16, fontweight='bo
plt.tight_layout()
plt.show()

# Figure properties and customization
print("\nFigure Properties and Customization:")
fig, ax = plt.subplots(figsize=(10, 6))

# Plot data
x = np.linspace(0, 10, 100)
y = np.sin(x) * np.exp(-x/10)

ax.plot(x, y, 'b-', linewidth=2, label='Damped Sine Wave')
ax.fill_between(x, y, alpha=0.3, color='blue')

# Customize the plot
ax.set_title('Figure Properties and Customization', fontsize=16, fontweig
ax.set_xlabel('Time', fontsize=12)
ax.set_ylabel('Amplitude', fontsize=12)
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3)

# Set axis limits and ticks
ax.set_xlim(0, 10)
ax.set_ylim(-1, 1)
ax.set_xticks(np.arange(0, 11, 2))
ax.set_yticks(np.arange(-1, 1.1, 0.5))

# Add text annotation
ax.text(5, 0.5, 'Peak Value', fontsize=12, ha='center',
        bbox=dict(boxstyle='round', facecolor='yellow', alpha=0.7))

# Add arrow annotation
ax.annotate('Decay', xy=(8, 0.2), xytext=(6, 0.6),
            arrowprops=dict(arrowstyle='->', color='red', lw=2),
            fontsize=12, ha='center')

plt.tight_layout()
plt.show()

print("\n=== END OF FIGURE AND AXES MANAGEMENT ===")

```

4. Plot Customization and Styling

Professional Appearance

Creating publication-quality plots requires attention to styling details:

Color Management

- **Named colors:** 'red', 'blue', 'green', etc.
- **Hex colors:** '#FF5733', '#2E86AB'
- **RGB tuples:** (0.1, 0.2, 0.5)
- **Colormaps:** 'viridis', 'plasma', 'inferno', 'coolwarm'

Typography

- **Font families:** 'serif', 'sans-serif', 'monospace'
- **Font sizes:** title, labels, ticks, legend
- **Font weights:** 'normal', 'bold', 'light'
- **Text alignment:** 'left', 'center', 'right'

Line and Marker Styles

- **Line styles:** '-', '--', '-.', ':', 'None'
- **Markers:** 'o', 's', '^', 'v', 'D', '*', '+'
- **Line widths:** 0.5, 1, 2, 3, etc.
- **Marker sizes:** 1, 5, 10, 20, etc.

Grid and Background

- **Grid styles:** Major/minor grids, transparency
- **Background colors:** Face, edge colors
- **Spine customization:** Top, bottom, left, right

```
In [ ]: # 4. Plot Customization and Styling – Professional Appearance
print("=== PLOT CUSTOMIZATION AND STYLING ===")

# Color Management Examples
print("Color Management Examples:")
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
fig.suptitle('Color Management and Styling', fontsize=16, fontweight='bold')

# Generate data
x = np.linspace(0, 2*np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.sin(x + np.pi/4)
y4 = np.cos(x + np.pi/4)

# 1. Named Colors
axes[0, 0].plot(x, y1, color='red', linewidth=2, label='sin(x)')
```

```

axes[0, 0].plot(x, y2, color='blue', linewidth=2, label='cos(x)')
axes[0, 0].set_title('Named Colors')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# 2. Hex Colors
axes[0, 1].plot(x, y1, color='#FF5733', linewidth=2, label='sin(x)')
axes[0, 1].plot(x, y2, color='#2E86AB', linewidth=2, label='cos(x)')
axes[0, 1].set_title('Hex Colors')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# 3. RGB Tuples
axes[1, 0].plot(x, y1, color=(0.1, 0.2, 0.5), linewidth=2, label='sin(x)')
axes[1, 0].plot(x, y2, color=(0.5, 0.1, 0.2), linewidth=2, label='cos(x)')
axes[1, 0].set_title('RGB Tuples')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# 4. Colormap
colors = plt.cm.viridis(np.linspace(0, 1, 4))
axes[1, 1].plot(x, y1, color=colors[0], linewidth=2, label='sin(x)')
axes[1, 1].plot(x, y2, color=colors[1], linewidth=2, label='cos(x)')
axes[1, 1].plot(x, y3, color=colors[2], linewidth=2, label='sin(x +  $\pi/4$ )')
axes[1, 1].plot(x, y4, color=colors[3], linewidth=2, label='cos(x +  $\pi/4$ )')
axes[1, 1].set_title('Colormap (viridis)')
axes[1, 1].legend()
axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Typography and Font Management
print("\nTypography and Font Management:")
fig, ax = plt.subplots(figsize=(12, 8))

# Generate data
x = np.linspace(0, 10, 100)
y1 = np.sin(x)
y2 = np.cos(x)

# Plot with custom typography
ax.plot(x, y1, 'b--', linewidth=3, label='Sine Wave', alpha=0.8)
ax.plot(x, y2, 'r--', linewidth=3, label='Cosine Wave', alpha=0.8)

# Customize typography
ax.set_title('Typography and Font Management',
             fontsize=20, fontweight='bold', fontfamily='serif', pad=20)
ax.set_xlabel('Time (seconds)', fontsize=14, fontweight='bold', fontfamily='sans-serif')
ax.set_ylabel('Amplitude', fontsize=14, fontweight='bold', fontfamily='sans-serif')

# Customize legend
ax.legend(fontsize=12, fontfamily='monospace', frameon=True,
         fancybox=True, shadow=True, loc='upper right')

# Customize tick labels
ax.tick_params(axis='both', which='major', labelsize=12, fontfamily='sans-serif')
ax.set_xticks(np.arange(0, 11, 2))
ax.set_yticks(np.arange(-1, 1.1, 0.5))

```

```

# Add text annotations with different fonts
ax.text(2, 0.8, 'Peak Value', fontsize=14, fontweight='bold',
        fontfamily='serif', ha='center',
        bbox=dict(boxstyle='round,pad=0.5', facecolor='yellow', alpha=0.7)

ax.text(8, -0.8, 'Trough Value', fontsize=14, fontweight='bold',
        fontfamily='serif', ha='center',
        bbox=dict(boxstyle='round,pad=0.5', facecolor='lightblue', alpha=

ax.grid(True, alpha=0.3, linestyle='--', linewidth=0.5)
plt.tight_layout()
plt.show()

# Line and Marker Styles
print("\nLine and Marker Styles:")
fig, axes = plt.subplots(2, 2, figsize=(15, 10))
fig.suptitle('Line and Marker Styles', fontsize=16, fontweight='bold')

# Generate data
x = np.linspace(0, 10, 20)
y = np.sin(x)

# 1. Line Styles
line_styles = ['-', '---', '-.', ':']
colors = ['blue', 'red', 'green', 'purple']
for i, (style, color) in enumerate(zip(line_styles, colors)):
    axes[0, 0].plot(x, y + i*0.5, linestyle=style, color=color,
                    linewidth=2, label=f'Style: {style}')
axes[0, 0].set_title('Line Styles')
axes[0, 0].legend()
axes[0, 0].grid(True, alpha=0.3)

# 2. Marker Styles
markers = ['o', 's', '^', 'v', 'D', '*', '+', 'x']
colors = plt.cm.tab10(np.linspace(0, 1, len(markers)))
for i, (marker, color) in enumerate(zip(markers, colors)):
    axes[0, 1].plot(x, y + i*0.3, marker=marker, color=color,
                    markersize=8, linestyle='None', label=f'Marker: {mark
axes[0, 1].set_title('Marker Styles')
axes[0, 1].legend()
axes[0, 1].grid(True, alpha=0.3)

# 3. Line Widths
widths = [0.5, 1, 2, 3, 4]
colors = plt.cm.viridis(np.linspace(0, 1, len(widths)))
for i, (width, color) in enumerate(zip(widths, colors)):
    axes[1, 0].plot(x, y + i*0.3, linewidth=width, color=color,
                    label=f'Width: {width}')
axes[1, 0].set_title('Line Widths')
axes[1, 0].legend()
axes[1, 0].grid(True, alpha=0.3)

# 4. Marker Sizes
sizes = [5, 10, 15, 20, 25]
colors = plt.cm.plasma(np.linspace(0, 1, len(sizes)))
for i, (size, color) in enumerate(zip(sizes, colors)):
    axes[1, 1].plot(x, y + i*0.3, marker='o', markersize=size, color=colo
                    linestyle='None', label=f'Size: {size}')
axes[1, 1].set_title('Marker Sizes')
axes[1, 1].legend()

```

```

axes[1, 1].grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

# Advanced Styling – Professional Plot
print("\nAdvanced Styling – Professional Plot:")
fig, ax = plt.subplots(figsize=(12, 8))

# Generate data
x = np.linspace(0, 4*np.pi, 200)
y1 = np.sin(x) * np.exp(-x/10)
y2 = np.cos(x) * np.exp(-x/10)
y3 = np.sin(x + np.pi/4) * np.exp(-x/10)

# Plot with advanced styling
ax.plot(x, y1, color='#2E86AB', linewidth=3, label='Damped Sine', alpha=0)
ax.plot(x, y2, color='#A23B72', linewidth=3, label='Damped Cosine', alpha=0)
ax.plot(x, y3, color='#F18F01', linewidth=3, label='Phase Shifted Sine',

# Fill between curves
ax.fill_between(x, y1, alpha=0.2, color='#2E86AB')
ax.fill_between(x, y2, alpha=0.2, color='#A23B72')

# Customize the plot
ax.set_title('Professional Damped Oscillations',
             fontsize=18, fontweight='bold', fontfamily='serif', pad=20)
ax.set_xlabel('Time (s)', fontsize=14, fontweight='bold')
ax.set_ylabel('Amplitude', fontsize=14, fontweight='bold')

# Customize legend
ax.legend(fontsize=12, frameon=True, fancybox=True, shadow=True,
          loc='upper right', framealpha=0.9)

# Customize grid
ax.grid(True, alpha=0.3, linestyle='--', linewidth=0.5, color='gray')
ax.set_axisbelow(True)

# Customize spines
for spine in ax.spines.values():
    spine.set_linewidth(1.5)
    spine.set_color('black')

# Set axis limits and ticks
ax.set_xlim(0, 4*np.pi)
ax.set_ylim(-1.2, 1.2)
ax.set_xticks(np.arange(0, 4*np.pi + 1, np.pi))
ax.set_xticklabels(['0', ' $\pi$ ', '2 $\pi$ ', '3 $\pi$ ', '4 $\pi$ '])
ax.set_yticks(np.arange(-1, 1.1, 0.5))

# Add annotations
ax.annotate('Peak', xy=(np.pi/2, 1), xytext=(np.pi/2 + 1, 1.1),
            arrowprops=dict(arrowstyle='->', color='red', lw=2),
            fontsize=12, ha='center', fontweight='bold')

# Add text box
ax.text(0.02, 0.98, 'Damped Oscillations\nwith Phase Shift',
        transform=ax.transAxes, fontsize=12, fontweight='bold',
        verticalalignment='top', bbox=dict(boxstyle='round,pad=0.5',
        facecolor='lightblue', alpha=0.8))

```

```
plt.tight_layout()
plt.show()

print("\n=== END OF PLOT CUSTOMIZATION AND STYLING ===")
```

5. Different Plot Types

Comprehensive Plot Gallery

Matplotlib offers a wide variety of plot types for different data visualization needs:

Statistical Plots

- **Histograms:** Distribution of numerical data
- **Box plots:** Statistical summaries and outliers
- **Violin plots:** Distribution shapes
- **Error bars:** Uncertainty visualization
- **Contour plots:** 3D data in 2D

Categorical Plots

- **Bar charts:** Categorical comparisons
- **Horizontal bar charts:** Long category names
- **Stacked bar charts:** Part-to-whole relationships
- **Grouped bar charts:** Multiple categories

Time Series Plots

- **Line plots:** Trends over time
- **Area plots:** Cumulative values
- **Stacked area plots:** Multiple time series
- **Candlestick plots:** Financial data

2D and 3D Plots

- **Scatter plots:** Two-variable relationships
- **Bubble charts:** Three-variable relationships
- **Heatmaps:** Matrix data visualization
- **3D surface plots:** Three-dimensional data

```
In [ ]: # 5. Different Plot Types - Comprehensive Gallery
print("=== DIFFERENT PLOT TYPES ===")

# Generate comprehensive sample data
np.random.seed(42)
n_samples = 1000

# Statistical data
data_normal = np.random.normal(0, 1, n_samples)
```

```

data_exponential = np.random.exponential(2, n_samples)
data_uniform = np.random.uniform(-2, 2, n_samples)

# Categorical data
categories = ['A', 'B', 'C', 'D', 'E']
values1 = [23, 45, 56, 78, 32]
values2 = [15, 25, 35, 45, 55]
values3 = [10, 20, 30, 40, 50]

# Time series data
time = np.linspace(0, 10, 100)
ts1 = np.sin(time) + 0.1 * np.random.randn(100)
ts2 = np.cos(time) + 0.1 * np.random.randn(100)
ts3 = np.sin(time + np.pi/4) + 0.1 * np.random.randn(100)

# 2D data for scatter and contour
x = np.linspace(-3, 3, 50)
y = np.linspace(-3, 3, 50)
X, Y = np.meshgrid(x, y)
Z = np.sin(X) * np.cos(Y)

# Create comprehensive plot gallery
fig = plt.figure(figsize=(20, 15))
fig.suptitle('Comprehensive Plot Types Gallery', fontsize=20, fontweight=

# 1. Histogram
ax1 = plt.subplot(3, 4, 1)
ax1.hist(data_normal, bins=30, alpha=0.7, color='skyblue', edgecolor='bla
ax1.set_title('Histogram')
ax1.set_xlabel('Value')
ax1.set_ylabel('Frequency')
ax1.grid(True, alpha=0.3)

# 2. Box Plot
ax2 = plt.subplot(3, 4, 2)
box_data = [data_normal, data_exponential, data_uniform]
ax2.boxplot(box_data, labels=['Normal', 'Exponential', 'Uniform'])
ax2.set_title('Box Plot')
ax2.set_ylabel('Value')
ax2.grid(True, alpha=0.3)

# 3. Bar Chart
ax3 = plt.subplot(3, 4, 3)
ax3.bar(categories, values1, color=['red', 'green', 'blue', 'orange', 'pu
ax3.set_title('Bar Chart')
ax3.set_xlabel('Categories')
ax3.set_ylabel('Values')
ax3.grid(True, alpha=0.3)

# 4. Scatter Plot
ax4 = plt.subplot(3, 4, 4)
x_scatter = np.random.randn(100)
y_scatter = 2 * x_scatter + np.random.randn(100) * 0.5
colors = np.random.rand(100)
ax4.scatter(x_scatter, y_scatter, c=colors, alpha=0.6, s=50)
ax4.set_title('Scatter Plot')
ax4.set_xlabel('X')
ax4.set_ylabel('Y')

# 5. Line Plot

```



```

ax5 = plt.subplot(3, 4, 5)
ax5.plot(time, ts1, 'b-', linewidth=2, label='Series 1')
ax5.plot(time, ts2, 'r-', linewidth=2, label='Series 2')
ax5.set_title('Line Plot')
ax5.set_xlabel('Time')
ax5.set_ylabel('Value')
ax5.legend()
ax5.grid(True, alpha=0.3)

# 6. Area Plot
ax6 = plt.subplot(3, 4, 6)
ax6.fill_between(time, ts1, alpha=0.7, color='blue', label='Series 1')
ax6.fill_between(time, ts2, alpha=0.7, color='red', label='Series 2')
ax6.set_title('Area Plot')
ax6.set_xlabel('Time')
ax6.set_ylabel('Value')
ax6.legend()
ax6.grid(True, alpha=0.3)

# 7. Pie Chart
ax7 = plt.subplot(3, 4, 7)
sizes = [30, 25, 20, 15, 10]
labels = ['A', 'B', 'C', 'D', 'E']
colors = ['gold', 'lightcoral', 'lightskyblue', 'lightgreen', 'plum']
ax7.pie(sizes, labels=labels, colors=colors, autopct='%1.1f%%', startangle=90)
ax7.set_title('Pie Chart')

# 8. Horizontal Bar Chart
ax8 = plt.subplot(3, 4, 8)
ax8.barh(categories, values1, color=['red', 'green', 'blue', 'orange', 'purple'])
ax8.set_title('Horizontal Bar Chart')
ax8.set_xlabel('Values')
ax8.set_ylabel('Categories')
ax8.grid(True, alpha=0.3)

# 9. Stacked Bar Chart
ax9 = plt.subplot(3, 4, 9)
width = 0.35
x_pos = np.arange(len(categories))
ax9.bar(x_pos, values1, width, label='Series 1', color='skyblue')
ax9.bar(x_pos, values2, width, bottom=values1, label='Series 2', color='lightcoral')
ax9.bar(x_pos, values3, width, bottom=np.array(values1) + np.array(values2),
        label='Series 3', color='lightgreen')
ax9.set_title('Stacked Bar Chart')
ax9.set_xlabel('Categories')
ax9.set_ylabel('Values')
ax9.set_xticks(x_pos)
ax9.set_xticklabels(categories)
ax9.legend()
ax9.grid(True, alpha=0.3)

# 10. Error Bar Plot
ax10 = plt.subplot(3, 4, 10)
x_error = np.arange(len(categories))
y_error = values1
yerr = [5, 3, 4, 6, 2] # Error values
ax10.errorbar(x_error, y_error, yerr=yerr, fmt='o', capsize=5, capthick=2)
ax10.set_title('Error Bar Plot')
ax10.set_xlabel('Categories')
ax10.set_ylabel('Values')

```

```

ax10.set_xticks(x_error)
ax10.set_xticklabels(categories)
ax10.grid(True, alpha=0.3)

# 11. Contour Plot
ax11 = plt.subplot(3, 4, 11)
contour = ax11.contour(X, Y, Z, levels=20)
ax11.clabel(contour, inline=True, fontsize=8)
ax11.set_title('Contour Plot')
ax11.set_xlabel('X')
ax11.set_ylabel('Y')

# 12. Heatmap
ax12 = plt.subplot(3, 4, 12)
heatmap_data = np.random.randn(10, 10)
im = ax12.imshow(heatmap_data, cmap='viridis', aspect='auto')
ax12.set_title('Heatmap')
plt.colorbar(im, ax=ax12)

plt.tight_layout()
plt.show()

# Advanced Plot Types
print("\nAdvanced Plot Types:")

# 3D Surface Plot
from mpl_toolkits.mplot3d import Axes3D

fig = plt.figure(figsize=(15, 5))

# 3D Surface
ax1 = fig.add_subplot(1, 3, 1, projection='3d')
ax1.plot_surface(X, Y, Z, cmap='viridis', alpha=0.8)
ax1.set_title('3D Surface Plot')
ax1.set_xlabel('X')
ax1.set_ylabel('Y')
ax1.set_zlabel('Z')

# 3D Scatter
ax2 = fig.add_subplot(1, 3, 2, projection='3d')
x_3d = np.random.randn(100)
y_3d = np.random.randn(100)
z_3d = np.random.randn(100)
colors_3d = np.random.rand(100)
ax2.scatter(x_3d, y_3d, z_3d, c=colors_3d, cmap='plasma')
ax2.set_title('3D Scatter Plot')
ax2.set_xlabel('X')
ax2.set_ylabel('Y')
ax2.set_zlabel('Z')

# 3D Line
ax3 = fig.add_subplot(1, 3, 3, projection='3d')
t = np.linspace(0, 4*np.pi, 100)
x_3d_line = np.sin(t)
y_3d_line = np.cos(t)
z_3d_line = t
ax3.plot(x_3d_line, y_3d_line, z_3d_line, 'b-', linewidth=2)
ax3.set_title('3D Line Plot')
ax3.set_xlabel('X')
ax3.set_ylabel('Y')

```

```

ax3.set_zlabel('Z')

plt.tight_layout()
plt.show()

# Financial Plots (Candlestick-style)
print("\nFinancial-style Plots:")

# Generate financial data
dates = pd.date_range('2023-01-01', periods=100, freq='D')
np.random.seed(42)
prices = 100 + np.cumsum(np.random.randn(100) * 0.5)
high = prices + np.random.uniform(0, 2, 100)
low = prices - np.random.uniform(0, 2, 100)
volume = np.random.uniform(1000, 5000, 100)

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(12, 8), height_ratios=[3, 1])

# Price chart
ax1.plot(dates, prices, 'b-', linewidth=2, label='Price')
ax1.fill_between(dates, low, high, alpha=0.3, color='blue', label='High-Low')
ax1.set_title('Financial Data - Price Chart')
ax1.set_ylabel('Price ($)')
ax1.legend()
ax1.grid(True, alpha=0.3)

# Volume chart
ax2.bar(dates, volume, alpha=0.7, color='orange')
ax2.set_title('Volume')
ax2.set_xlabel('Date')
ax2.set_ylabel('Volume')
ax2.grid(True, alpha=0.3)

plt.tight_layout()
plt.show()

print("\n=== END OF DIFFERENT PLOT TYPES ===")

```

6. Introduction to Seaborn

What is Seaborn?

Seaborn is a high-level statistical data visualization library built on top of Matplotlib. It provides a more convenient interface for creating attractive and informative statistical graphics.

Key Advantages

- **Statistical focus:** Built-in statistical functions and distributions
- **Beautiful defaults:** Attractive color palettes and styling
- **Pandas integration:** Works seamlessly with DataFrames
- **High-level functions:** Complex plots with simple commands
- **Automatic estimation:** Built-in statistical estimation and error bars

Seaborn vs Matplotlib

- **Matplotlib:** Low-level control, basic plots, customization
- **Seaborn:** High-level statistical plots, beautiful defaults, less customization

Seaborn Categories

- **Distribution plots:** histplot, kdeplot, rugplot
- **Categorical plots:** boxplot, violinplot, stripplot, swarmplot
- **Relational plots:** scatterplot, lineplot, relplot
- **Matrix plots:** heatmap, clustermap
- **Regression plots:** regplot, lmpplot

```
In [ ]: # 6. Introduction to Seaborn – Statistical Visualization
print("=== INTRODUCTION TO SEABORN ===")

# Set Seaborn style
sns.set_style("whitegrid")
sns.set_palette("husl")

# Generate sample data
np.random.seed(42)
n_samples = 1000

# Create a comprehensive dataset
data = {
    'x': np.random.randn(n_samples),
    'y': 2 * np.random.randn(n_samples) + 1,
    'category': np.random.choice(['A', 'B', 'C', 'D'], n_samples),
    'value': np.random.exponential(2, n_samples),
    'group': np.random.choice(['Group1', 'Group2', 'Group3'], n_samples)
}
df = pd.DataFrame(data)

print("Sample Dataset:")
print(df.head())
print(f"\nDataset shape: {df.shape}")
print(f"Columns: {df.columns.tolist()}")

# Basic Seaborn plots
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Introduction to Seaborn – Basic Plots', fontsize=16, fontwe

# 1. Histogram with KDE
sns.histplot(data=df, x='x', kde=True, ax=axes[0, 0])
axes[0, 0].set_title('Histogram with KDE')

# 2. Scatter plot
sns.scatterplot(data=df, x='x', y='y', hue='category', ax=axes[0, 1])
axes[0, 1].set_title('Scatter Plot with Hue')

# 3. Box plot
sns.boxplot(data=df, x='category', y='value', ax=axes[0, 2])
axes[0, 2].set_title('Box Plot')

# 4. Violin plot
sns.violinplot(data=df, x='category', y='value', ax=axes[1, 0])
```

```

axes[1, 0].set_title('Violin Plot')

# 5. Heatmap
correlation_data = df[['x', 'y', 'value']].corr()
sns.heatmap(correlation_data, annot=True, cmap='coolwarm', center=0, ax=axes[1, 1])
axes[1, 1].set_title('Correlation Heatmap')

# 6. Line plot
sns.lineplot(data=df.groupby('category')['value'].mean().reset_index(),
              x='category', y='value', ax=axes[1, 2])
axes[1, 2].set_title('Line Plot (Mean Values)')

plt.tight_layout()
plt.show()

# Seaborn styling and themes
print("\nSeaborn Styling and Themes:")

# Different styles
styles = ['darkgrid', 'whitegrid', 'dark', 'white', 'ticks']
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Seaborn Styles', fontsize=16, fontweight='bold')

for i, style in enumerate(styles):
    sns.set_style(style)
    row = i // 3
    col = i % 3

    if i < len(styles):
        sns.histplot(data=df, x='x', kde=True, ax=axes[row, col])
        axes[row, col].set_title(f'Style: {style}')

# Remove the last empty subplot
if len(styles) < 6:
    axes[1, 2].remove()

plt.tight_layout()
plt.show()

# Reset to default style
sns.set_style("whitegrid")

# Color palettes
print("\nSeaborn Color Palettes:")

palettes = ['deep', 'muted', 'pastel', 'bright', 'dark', 'colorblind']
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Seaborn Color Palettes', fontsize=16, fontweight='bold')

for i, palette in enumerate(palettes):
    row = i // 3
    col = i % 3

    sns.set_palette(palette)
    sns.scatterplot(data=df, x='x', y='y', hue='category', ax=axes[row, col])
    axes[row, col].set_title(f'Palette: {palette}')

plt.tight_layout()
plt.show()

```

```

# Reset to default palette
sns.set_palette("husl")

# Seaborn with Pandas DataFrames
print("\nSeaborn with Pandas DataFrames:")

# Create a more complex dataset
np.random.seed(42)
n_samples = 500

complex_data = {
    'age': np.random.normal(35, 10, n_samples),
    'income': np.random.normal(50000, 15000, n_samples),
    'education': np.random.choice(['High School', 'Bachelor', 'Master', 'PhD'], n_samples),
    'experience': np.random.normal(10, 5, n_samples),
    'department': np.random.choice(['IT', 'HR', 'Finance', 'Marketing'], n_samples),
    'performance': np.random.normal(7, 2, n_samples)
}

df_complex = pd.DataFrame(complex_data)
df_complex['age'] = np.clip(df_complex['age'], 22, 65) # Clip age to realistic range
df_complex['experience'] = np.clip(df_complex['experience'], 0, 40) # Clip experience
df_complex['performance'] = np.clip(df_complex['performance'], 1, 10) # Clip performance

print("Complex Dataset Sample:")
print(df_complex.head())

# Create comprehensive Seaborn visualizations
fig, axes = plt.subplots(2, 3, figsize=(20, 12))
fig.suptitle('Seaborn with Complex Data', fontsize=16, fontweight='bold')

# 1. Distribution plot
sns.histplot(data=df_complex, x='age', hue='education', kde=True, ax=axes[0, 0])
axes[0, 0].set_title('Age Distribution by Education')

# 2. Scatter plot with regression
sns.scatterplot(data=df_complex, x='age', y='income', hue='education', ax=axes[0, 1])
sns.regplot(data=df_complex, x='age', y='income', scatter=False, ax=axes[0, 1])
axes[0, 1].set_title('Age vs Income with Regression')

# 3. Box plot
sns.boxplot(data=df_complex, x='department', y='performance', hue='education', ax=axes[0, 2])
axes[0, 2].set_title('Performance by Department and Education')
axes[0, 2].tick_params(axis='x', rotation=45)

# 4. Violin plot
sns.violinplot(data=df_complex, x='education', y='income', ax=axes[1, 0])
axes[1, 0].set_title('Income Distribution by Education')
axes[1, 0].tick_params(axis='x', rotation=45)

# 5. Heatmap
pivot_data = df_complex.groupby(['department', 'education'])['performance'].mean().reset_index()
sns.heatmap(pivot_data, annot=True, cmap='YlOrRd', ax=axes[1, 1])
axes[1, 1].set_title('Performance Heatmap')

# 6. Pair plot (simplified)
sns.scatterplot(data=df_complex, x='age', y='experience', hue='department', ax=axes[1, 2])
axes[1, 2].set_title('Age vs Experience by Department')

plt.tight_layout()

```

```
plt.show()

print("\n=== END OF SEABORN INTRODUCTION ===")
```

7. Statistical Plots and Distributions

Distribution Visualization

Seaborn excels at creating statistical plots that reveal data distributions and relationships:

Distribution Plots

- `histplot()` : Histograms with optional KDE overlay
- `kdeplot()` : Kernel Density Estimation plots
- `rugplot()` : Rug plots for showing individual data points
- `ecdfplot()` : Empirical Cumulative Distribution Function

Statistical Estimation

- `regplot()` : Linear regression with confidence intervals
- `residplot()` : Residual plots for regression analysis
- `distplot()` : Combined histogram and KDE (deprecated, use `histplot`)

Distribution Comparison

- **Multiple distributions:** Overlay different distributions
- **Faceted plots:** Separate plots for different categories
- **Statistical tests:** Built-in statistical comparisons

```
In [ ]: # 7. Statistical Plots and Distributions
print("=== STATISTICAL PLOTS AND DISTRIBUTIONS ===")

# Generate comprehensive statistical data
np.random.seed(42)
n_samples = 1000

# Create different distributions
normal_data = np.random.normal(0, 1, n_samples)
exponential_data = np.random.exponential(2, n_samples)
uniform_data = np.random.uniform(-2, 2, n_samples)
gamma_data = np.random.gamma(2, 1, n_samples)

# Create DataFrame for Seaborn
dist_data = pd.DataFrame({
    'normal': normal_data,
    'exponential': exponential_data,
    'uniform': uniform_data,
    'gamma': gamma_data
})

# Melt for easier plotting
```

```

dist_data_melted = dist_data.melt(var_name='distribution', value_name='value')

print("Distribution Data Sample:")
print(dist_data.head())

# Distribution plots
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Statistical Plots and Distributions', fontsize=16, fontweight='bold')

# 1. Histogram with KDE
sns.histplot(data=dist_data_melted, x='value', hue='distribution', kde=True)
axes[0, 0].set_title('Histogram with KDE')

# 2. KDE Plot
sns.kdeplot(data=dist_data_melted, x='value', hue='distribution', ax=axes[0, 1])
axes[0, 1].set_title('KDE Plot')

# 3. ECDF Plot
sns.ecdfplot(data=dist_data_melted, x='value', hue='distribution', ax=axes[0, 2])
axes[0, 2].set_title('ECDF Plot')

# 4. Rug Plot
sns.rugplot(data=dist_data_melted, x='value', hue='distribution', ax=axes[1, 0])
axes[1, 0].set_title('Rug Plot')

# 5. Box Plot
sns.boxplot(data=dist_data_melted, x='distribution', y='value', ax=axes[1, 1])
axes[1, 1].set_title('Box Plot')
axes[1, 1].tick_params(axis='x', rotation=45)

# 6. Violin Plot
sns.violinplot(data=dist_data_melted, x='distribution', y='value', ax=axes[1, 2])
axes[1, 2].set_title('Violin Plot')
axes[1, 2].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

# Regression and correlation analysis
print("\nRegression and Correlation Analysis:")

# Generate correlated data
np.random.seed(42)
n_samples = 200
x = np.random.normal(0, 1, n_samples)
y = 2 * x + np.random.normal(0, 0.5, n_samples)
z = x + y + np.random.normal(0, 0.3, n_samples)

regression_data = pd.DataFrame({
    'x': x,
    'y': y,
    'z': z,
    'category': np.random.choice(['A', 'B', 'C'], n_samples)
})

fig, axes = plt.subplots(2, 2, figsize=(15, 12))
fig.suptitle('Regression and Correlation Analysis', fontsize=16, fontweight='bold')

# 1. Scatter plot with regression line
sns.scatterplot(data=regression_data, x='x', y='y', ax=axes[0, 0])

```



```

sns.regplot(data=regression_data, x='x', y='y', ax=axes[0, 0])
axes[0, 0].set_title('Scatter Plot with Regression')

# 2. Residual plot
sns.residplot(data=regression_data, x='x', y='y', ax=axes[0, 1])
axes[0, 1].set_title('Residual Plot')

# 3. Correlation heatmap
correlation_matrix = regression_data[['x', 'y', 'z']].corr()
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0, ax=
axes[1, 0].set_title('Correlation Heatmap')

# 4. Scatter plot with hue
sns.scatterplot(data=regression_data, x='x', y='y', hue='category', ax=ax
sns.regplot(data=regression_data, x='x', y='y', scatter=False, ax=axes[1,
axes[1, 1].set_title('Scatter Plot with Categories')

plt.tight_layout()
plt.show()

# Advanced statistical plots
print("\nAdvanced Statistical Plots:")

# Create a more complex dataset
np.random.seed(42)
n_samples = 500

advanced_data = {
    'value1': np.random.normal(0, 1, n_samples),
    'value2': np.random.normal(1, 1.5, n_samples),
    'value3': np.random.normal(2, 0.8, n_samples),
    'category': np.random.choice(['Group A', 'Group B', 'Group C'], n_sam
    'subcategory': np.random.choice(['Type 1', 'Type 2', 'Type 3'], n_sam
}

df_advanced = pd.DataFrame(advanced_data)

# Create advanced statistical visualizations
fig, axes = plt.subplots(2, 3, figsize=(20, 12))
fig.suptitle('Advanced Statistical Plots', fontsize=16, fontweight='bold')

# 1. Distribution comparison
sns.histplot(data=df_advanced, x='value1', hue='category', kde=True, ax=a
axes[0, 0].set_title('Distribution Comparison')

# 2. Multiple distributions
sns.kdeplot(data=df_advanced, x='value1', hue='category', ax=axes[0, 1])
sns.kdeplot(data=df_advanced, x='value2', hue='category', ax=axes[0, 1])
axes[0, 1].set_title('Multiple Distributions')

# 3. Faceted histogram
sns.histplot(data=df_advanced, x='value1', hue='category', kde=True,
             multiple='stack', ax=axes[0, 2])
axes[0, 2].set_title('Stacked Histogram')

# 4. Violin plot with split
sns.violinplot(data=df_advanced, x='category', y='value1', hue='subcatego
               split=True, ax=axes[1, 0])
axes[1, 0].set_title('Split Violin Plot')
axes[1, 0].tick_params(axis='x', rotation=45)

```

```

# 5. Box plot with swarm
sns.boxplot(data=df_advanced, x='category', y='value1', ax=axes[1, 1])
sns.swarmplot(data=df_advanced, x='category', y='value1', color='black',
axes[1, 1].set_title('Box Plot with Swarm')
axes[1, 1].tick_params(axis='x', rotation=45)

# 6. Strip plot
sns.stripplot(data=df_advanced, x='category', y='value1', hue='subcategor
ax=axes[1, 2])
axes[1, 2].set_title('Strip Plot')
axes[1, 2].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

# Statistical summary
print("\nStatistical Summary:")
print("Distribution Statistics:")
print(dist_data.describe())

print("\nCorrelation Matrix:")
print(regression_data[['x', 'y', 'z']].corr())

print("\n=== END OF STATISTICAL PLOTS AND DISTRIBUTIONS ===")

```

8. Categorical Plots

Visualizing Categorical Data

Seaborn provides specialized functions for visualizing categorical data effectively:

Box Plots

- **boxplot()** : Statistical summaries with quartiles and outliers
- **boxenplot()** : Letter-value plots for large datasets
- **Customization**: Colors, orientations, grouping

Violin Plots

- **violinplot()** : Distribution shapes with box plot overlay
- **Split violins**: Compare distributions across categories
- **Inner plots**: Box plots, quartiles, or points inside violins

Strip and Swarm Plots

- **stripplot()** : Individual data points with jitter
- **swarmplot()** : Non-overlapping points with automatic spacing
- **Combined plots**: Overlay on box plots for detailed view

Bar Plots

- **barplot()** : Point estimates with confidence intervals
- **countplot()** : Counts of categorical variables
- **catplot()** : Faceted categorical plots

```
In [ ]: # 8. Categorical Plots – Visualizing Categorical Data
print("=== CATEGORICAL PLOTS ===")

# Generate comprehensive categorical data
np.random.seed(42)
n_samples = 500

# Create a complex categorical dataset
categorical_data = {
    'value': np.concatenate([
        np.random.normal(0, 1, 100),      # Group A
        np.random.normal(1, 1.5, 100),    # Group B
        np.random.normal(2, 0.8, 100),    # Group C
        np.random.normal(1.5, 1.2, 100),  # Group D
        np.random.normal(0.5, 2, 100)     # Group E
    ]),
    'category': ['Group A'] * 100 + ['Group B'] * 100 + ['Group C'] * 100 +
                ['Group D'] * 100 + ['Group E'] * 100,
    'subcategory': np.random.choice(['Type 1', 'Type 2', 'Type 3'], n_samples),
    'score': np.random.uniform(1, 10, n_samples),
    'count': np.random.poisson(5, n_samples)
}

df_categorical = pd.DataFrame(categorical_data)

print("Categorical Dataset Sample:")
print(df_categorical.head())
print(f"\nDataset shape: {df_categorical.shape}")

# Basic categorical plots
fig, axes = plt.subplots(2, 3, figsize=(20, 12))
fig.suptitle('Categorical Plots – Basic Types', fontsize=16, fontweight='bold')

# 1. Box Plot
sns.boxplot(data=df_categorical, x='category', y='value', ax=axes[0, 0])
axes[0, 0].set_title('Box Plot')
axes[0, 0].tick_params(axis='x', rotation=45)

# 2. Violin Plot
sns.violinplot(data=df_categorical, x='category', y='value', ax=axes[0, 1])
axes[0, 1].set_title('Violin Plot')
axes[0, 1].tick_params(axis='x', rotation=45)

# 3. Strip Plot
sns.stripplot(data=df_categorical, x='category', y='value', ax=axes[0, 2])
axes[0, 2].set_title('Strip Plot')
axes[0, 2].tick_params(axis='x', rotation=45)

# 4. Swarm Plot
sns.swarmplot(data=df_categorical, x='category', y='value', ax=axes[1, 0])
axes[1, 0].set_title('Swarm Plot')
axes[1, 0].tick_params(axis='x', rotation=45)

# 5. Bar Plot
sns.barplot(data=df_categorical, x='category', y='value', ax=axes[1, 1])
```

```

axes[1, 1].set_title('Bar Plot')
axes[1, 1].tick_params(axis='x', rotation=45)

# 6. Count Plot
sns.countplot(data=df_categorical, x='category', ax=axes[1, 2])
axes[1, 2].set_title('Count Plot')
axes[1, 2].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

# Advanced categorical plots
print("\nAdvanced Categorical Plots:")

fig, axes = plt.subplots(2, 3, figsize=(20, 12))
fig.suptitle('Advanced Categorical Plots', fontsize=16, fontweight='bold')

# 1. Box plot with hue
sns.boxplot(data=df_categorical, x='category', y='value', hue='subcategory')
axes[0, 0].set_title('Box Plot with Hue')
axes[0, 0].tick_params(axis='x', rotation=45)

# 2. Violin plot with split
sns.violinplot(data=df_categorical, x='category', y='value', hue='subcategory',
               split=True, ax=axes[0, 1])
axes[0, 1].set_title('Split Violin Plot')
axes[0, 1].tick_params(axis='x', rotation=45)

# 3. Box plot with swarm overlay
sns.boxplot(data=df_categorical, x='category', y='value', ax=axes[0, 2])
sns.swarmplot(data=df_categorical, x='category', y='value', color='black', ax=axes[0, 2])
axes[0, 2].set_title('Box Plot with Swarm Overlay')
axes[0, 2].tick_params(axis='x', rotation=45)

# 4. Horizontal box plot
sns.boxplot(data=df_categorical, y='category', x='value', ax=axes[1, 0])
axes[1, 0].set_title('Horizontal Box Plot')

# 5. Boxen plot (letter-value plot)
sns.boxenplot(data=df_categorical, x='category', y='value', ax=axes[1, 1])
axes[1, 1].set_title('Boxen Plot (Letter-Value)')
axes[1, 1].tick_params(axis='x', rotation=45)

# 6. Point plot
sns.pointplot(data=df_categorical, x='category', y='value', hue='subcategory')
axes[1, 2].set_title('Point Plot')
axes[1, 2].tick_params(axis='x', rotation=45)

plt.tight_layout()
plt.show()

# Faceted categorical plots
print("\nFaceted Categorical Plots:")

# Create a faceted plot using catplot
g = sns.catplot(data=df_categorical, x='category', y='value', hue='subcategory',
               col='subcategory', kind='box', height=4, aspect=0.8)
g.fig.suptitle('Faceted Box Plots by Subcategory', y=1.02, fontsize=16, fontweight='bold')
plt.show()

```

```

# Custom categorical plot
fig, ax = plt.subplots(figsize=(12, 8))

# Create a custom categorical visualization
sns.violinplot(data=df_categorical, x='category', y='value', hue='subcate
               split=True, inner='quart', ax=ax)
ax.set_title('Custom Violin Plot with Quartiles', fontsize=16, fontweight
ax.set_xlabel('Category', fontsize=12)
ax.set_ylabel('Value', fontsize=12)
ax.tick_params(axis='x', rotation=45)
ax.legend(title='Subcategory', bbox_to_anchor=(1.05, 1), loc='upper left')

plt.tight_layout()
plt.show()

# Statistical summary by category
print("\nStatistical Summary by Category:")
print(df_categorical.groupby('category')['value'].describe())

print("\n=== END OF CATEGORICAL PLOTS ===")

```

9. Matrix Plots and Heatmaps

Visualizing Matrix Data

Matrix plots are essential for visualizing relationships and patterns in structured data:

Heatmaps

- **heatmap()** : Color-coded matrix visualization
- **Correlation matrices**: Show relationships between variables
- **Annotated heatmaps**: Add numerical values to cells
- **Custom colormaps**: Choose appropriate color schemes

Clustermaps

- **clustermap()** : Hierarchical clustering with heatmap
- **Row/column clustering**: Group similar rows or columns
- **Dendrograms**: Show clustering hierarchy
- **Custom clustering**: Use different distance metrics

Matrix Plot Types

- **Correlation heatmaps**: Variable relationships
- **Confusion matrices**: Classification results
- **Distance matrices**: Similarity measures
- **Pivot table visualizations**: Aggregated data

```

In [ ]: # 9. Matrix Plots and Heatmaps – Visualizing Matrix Data
print("=== MATRIX PLOTS AND HEATMAPS ===")

# Generate comprehensive matrix data

```

```

np.random.seed(42)
n_samples = 200

# Create a dataset with multiple variables
matrix_data = {
    'var1': np.random.normal(0, 1, n_samples),
    'var2': np.random.normal(1, 1.5, n_samples),
    'var3': np.random.normal(2, 0.8, n_samples),
    'var4': np.random.normal(1.5, 1.2, n_samples),
    'var5': np.random.normal(0.5, 2, n_samples),
    'var6': np.random.normal(3, 1, n_samples),
    'category': np.random.choice(['A', 'B', 'C', 'D'], n_samples),
    'score': np.random.uniform(1, 10, n_samples)
}

df_matrix = pd.DataFrame(matrix_data)

# Calculate correlation matrix
correlation_matrix = df_matrix[['var1', 'var2', 'var3', 'var4', 'var5', 'var6', 'category', 'score'].corr()

print("Matrix Data Sample:")
print(df_matrix.head())
print(f"\nCorrelation Matrix:")
print(correlation_matrix)

# Basic heatmaps
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle('Matrix Plots and Heatmaps', fontsize=16, fontweight='bold')

# 1. Basic heatmap
sns.heatmap(correlation_matrix, ax=axes[0, 0])
axes[0, 0].set_title('Basic Heatmap')

# 2. Annotated heatmap
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm', center=0, ax=axes[0, 1])
axes[0, 1].set_title('Annotated Heatmap')

# 3. Custom colormap
sns.heatmap(correlation_matrix, annot=True, cmap='RdYlBu_r', center=0, square=True, ax=axes[1, 0])
axes[1, 0].set_title('Custom Colormap (RdYlBu)')

# 4. Masked heatmap
mask = np.triu(np.ones_like(correlation_matrix, dtype=bool))
sns.heatmap(correlation_matrix, mask=mask, annot=True, cmap='viridis', square=True, ax=axes[1, 1])
axes[1, 1].set_title('Masked Heatmap (Upper Triangle)')

plt.tight_layout()
plt.show()

# Advanced heatmaps
print("\nAdvanced Heatmaps:")

# Create a more complex dataset
np.random.seed(42)
n_samples = 300

# Generate data with some structure
advanced_matrix_data = {

```

```

'feature1': np.random.normal(0, 1, n_samples),
'feature2': np.random.normal(1, 1.5, n_samples),
'feature3': np.random.normal(2, 0.8, n_samples),
'feature4': np.random.normal(1.5, 1.2, n_samples),
'feature5': np.random.normal(0.5, 2, n_samples),
'feature6': np.random.normal(3, 1, n_samples),
'feature7': np.random.normal(1, 1.8, n_samples),
'feature8': np.random.normal(2.5, 0.9, n_samples),
'group': np.random.choice(['Group1', 'Group2', 'Group3'], n_samples),
'category': np.random.choice(['Type A', 'Type B', 'Type C'], n_sample
}

df_advanced_matrix = pd.DataFrame(advanced_matrix_data)

# Calculate correlation matrix
advanced_corr = df_advanced_matrix[['feature1', 'feature2', 'feature3', '
                                     'feature5', 'feature6', 'feature7', 'f

# Create advanced heatmap visualizations
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle('Advanced Heatmap Visualizations', fontsize=16, fontweight='

# 1. Clustered heatmap
sns.heatmap(advanced_corr, annot=True, cmap='coolwarm', center=0,
            square=True, ax=axes[0, 0])
axes[0, 0].set_title('Clustered Heatmap')

# 2. Diverging colormap
sns.heatmap(advanced_corr, annot=True, cmap='RdBu_r', center=0,
            square=True, ax=axes[0, 1])
axes[0, 1].set_title('Diverging Colormap')

# 3. Sequential colormap
sns.heatmap(advanced_corr, annot=True, cmap='YlOrRd',
            square=True, ax=axes[1, 0])
axes[1, 0].set_title('Sequential Colormap')

# 4. Custom formatting
sns.heatmap(advanced_corr, annot=True, cmap='viridis', center=0,
            square=True, fmt='.2f', ax=axes[1, 1])
axes[1, 1].set_title('Custom Formatting')

plt.tight_layout()
plt.show()

# Clustermap
print("\nClustermap:")

# Create a clustermap
g = sns.clustermap(advanced_corr, annot=True, cmap='coolwarm', center=0,
                  square=True, figsize=(10, 8))
g.fig.suptitle('Clustermap with Hierarchical Clustering', y=1.02, fontsiz
plt.show()

# Pivot table heatmap
print("\nPivot Table Heatmap:")

# Create a pivot table
pivot_data = df_advanced_matrix.groupby(['group', 'category'])['feature1'

```

```

fig, ax = plt.subplots(figsize=(10, 6))
sns.heatmap(pivot_data, annot=True, cmap='YlOrRd', ax=ax)
ax.set_title('Pivot Table Heatmap – Mean Feature1 by Group and Category')
plt.tight_layout()
plt.show()

# Distance matrix
print("\nDistance Matrix:")

# Calculate distance matrix
from scipy.spatial.distance import pdist, squareform
distance_matrix = squareform(pdist(advanced_corr))

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(distance_matrix, annot=True, cmap='viridis', ax=ax)
ax.set_title('Distance Matrix')
plt.tight_layout()
plt.show()

# Confusion matrix example
print("\nConfusion Matrix Example:")

# Generate confusion matrix data
np.random.seed(42)
confusion_data = np.random.randint(0, 4, (4, 4))
confusion_data = confusion_data + np.diag([10, 15, 12, 8]) # Add diagona

fig, ax = plt.subplots(figsize=(8, 6))
sns.heatmap(confusion_data, annot=True, cmap='Blues', ax=ax)
ax.set_title('Confusion Matrix')
ax.set_xlabel('Predicted')
ax.set_ylabel('Actual')
plt.tight_layout()
plt.show()

print("\n=== END OF MATRIX PLOTS AND HEATMAPS ===")

```

10. Subplots and Multiple Plots

Complex Layouts

Creating sophisticated multi-panel figures requires understanding subplot management:

Subplot Methods

- `plt.subplots()` : Create multiple subplots in a grid
- `plt.subplot()` : Add individual subplots
- `plt.subplot2grid()` : Create subplots with custom grid
- `GridSpec` : Advanced subplot layouts

Layout Strategies

- **Grid layouts:** Regular rows and columns
- **Custom layouts:** Irregular subplot arrangements

- **Nested layouts:** Subplots within subplots
- **Shared axes:** Common x or y axes across subplots

Multi-plot Types

- **Dashboard layouts:** Multiple related plots
- **Comparison plots:** Side-by-side comparisons
- **Detail views:** Overview with zoomed sections
- **Multi-panel figures:** Complex scientific figures

```
In [ ]: # 10. Subplots and Multiple Plots – Complex Layouts
print("=== SUBPLOTS AND MULTIPLE PLOTS ===")

# Generate comprehensive data for multiple plots
np.random.seed(42)
n_samples = 1000

# Create a complex dataset
multiplot_data = {
    'x': np.random.normal(0, 1, n_samples),
    'y': 2 * np.random.normal(0, 1, n_samples) + 1,
    'z': np.random.normal(0, 1, n_samples),
    'category': np.random.choice(['A', 'B', 'C', 'D'], n_samples),
    'group': np.random.choice(['Group1', 'Group2', 'Group3'], n_samples),
    'value': np.random.exponential(2, n_samples),
    'time': np.linspace(0, 10, n_samples),
    'trend': np.sin(np.linspace(0, 4*np.pi, n_samples)) + 0.1 * np.random
}

df_multiplot = pd.DataFrame(multiplot_data)

print("Multiplot Dataset Sample:")
print(df_multiplot.head())

# Basic subplot layouts
print("\nBasic Subplot Layouts:")

# 1. Simple grid layout
fig, axes = plt.subplots(2, 3, figsize=(18, 12))
fig.suptitle('Basic Subplot Layouts', fontsize=16, fontweight='bold')

# Plot different visualizations
axes[0, 0].hist(df_multiplot['x'], bins=30, alpha=0.7, color='skyblue')
axes[0, 0].set_title('Histogram')

axes[0, 1].scatter(df_multiplot['x'], df_multiplot['y'], alpha=0.6, c=df_
axes[0, 1].set_title('Scatter Plot')

axes[0, 2].boxplot([df_multiplot[df_multiplot['category'] == cat]['value'
axes[0, 2].set_title('Box Plot')
axes[0, 2].set_xticklabels(['A', 'B', 'C', 'D'])

axes[1, 0].plot(df_multiplot['time'], df_multiplot['trend'], linewidth=2)
axes[1, 0].set_title('Time Series')

axes[1, 1].bar(['A', 'B', 'C', 'D'], df_multiplot['category'].value_count
axes[1, 1].set_title('Bar Chart')
```

```

axes[1, 2].pie(df_multiplot['group'].value_counts().values, labels=df_multiplot['group'].value_counts().index)
axes[1, 2].set_title('Pie Chart')

plt.tight_layout()
plt.show()

# Advanced subplot layouts
print("\nAdvanced Subplot Layouts:")

# 2. Custom layout with subplot2grid
fig = plt.figure(figsize=(20, 12))
fig.suptitle('Advanced Subplot Layouts', fontsize=16, fontweight='bold')

# Main plot (large)
ax1 = plt.subplot2grid((3, 4), (0, 0), colspan=2, rowspan=2)
ax1.scatter(df_multiplot['x'], df_multiplot['y'], c=df_multiplot['value'])
ax1.set_title('Main Scatter Plot')
ax1.set_xlabel('X')
ax1.set_ylabel('Y')

# Small plots
ax2 = plt.subplot2grid((3, 4), (0, 2))
ax2.hist(df_multiplot['x'], bins=20, alpha=0.7, color='lightblue')
ax2.set_title('X Distribution')

ax3 = plt.subplot2grid((3, 4), (0, 3))
ax3.hist(df_multiplot['y'], bins=20, alpha=0.7, color='lightcoral')
ax3.set_title('Y Distribution')

ax4 = plt.subplot2grid((3, 4), (1, 2))
ax4.boxplot([df_multiplot[df_multiplot['category'] == cat]['value'] for cat in df_multiplot['category'].unique()])
ax4.set_title('Value by Category')
ax4.set_xticklabels(['A', 'B', 'C', 'D'])

ax5 = plt.subplot2grid((3, 4), (1, 3))
ax5.pie(df_multiplot['group'].value_counts().values, labels=df_multiplot['group'].value_counts().index)
ax5.set_title('Group Distribution')

# Bottom plot (spans full width)
ax6 = plt.subplot2grid((3, 4), (2, 0), colspan=4)
ax6.plot(df_multiplot['time'], df_multiplot['trend'], linewidth=2, color='red')
ax6.fill_between(df_multiplot['time'], df_multiplot['trend'], alpha=0.3, color='red')
ax6.set_title('Time Series Trend')
ax6.set_xlabel('Time')
ax6.set_ylabel('Value')

plt.tight_layout()
plt.show()

# GridSpec for complex layouts
print("\nGridSpec for Complex Layouts:")

from matplotlib.gridspec import GridSpec

fig = plt.figure(figsize=(20, 12))
gs = GridSpec(4, 4, figure=fig)
fig.suptitle('GridSpec Complex Layout', fontsize=16, fontweight='bold')

# Create axes with GridSpec

```

```

ax1 = fig.add_subplot(gs[0, :]) # Top row, all columns
ax2 = fig.add_subplot(gs[1, 0]) # Second row, first column
ax3 = fig.add_subplot(gs[1, 1]) # Second row, second column
ax4 = fig.add_subplot(gs[1, 2]) # Second row, third column
ax5 = fig.add_subplot(gs[1, 3]) # Second row, fourth column
ax6 = fig.add_subplot(gs[2, :2]) # Third row, first two columns
ax7 = fig.add_subplot(gs[2, 2:]) # Third row, last two columns
ax8 = fig.add_subplot(gs[3, :]) # Bottom row, all columns

# Plot on each axis
ax1.plot(df_multiplot['time'], df_multiplot['trend'], linewidth=2, color=
ax1.set_title('Time Series Overview')

ax2.hist(df_multiplot['x'], bins=20, alpha=0.7, color='skyblue')
ax2.set_title('X Histogram')

ax3.hist(df_multiplot['y'], bins=20, alpha=0.7, color='lightcoral')
ax3.set_title('Y Histogram')

ax4.hist(df_multiplot['z'], bins=20, alpha=0.7, color='lightgreen')
ax4.set_title('Z Histogram')

ax5.hist(df_multiplot['value'], bins=20, alpha=0.7, color='gold')
ax5.set_title('Value Histogram')

ax6.scatter(df_multiplot['x'], df_multiplot['y'], alpha=0.6, c=df_multiplot['value'])
ax6.set_title('X vs Y Scatter')

ax7.scatter(df_multiplot['x'], df_multiplot['z'], alpha=0.6, c=df_multiplot['value'])
ax7.set_title('X vs Z Scatter')

ax8.boxplot([df_multiplot[df_multiplot['category'] == cat]['value'] for cat in categories])
ax8.set_title('Value by Category')
ax8.set_xticklabels(['A', 'B', 'C', 'D'])

plt.tight_layout()
plt.show()

# Shared axes
print("\nShared Axes:")

fig, axes = plt.subplots(2, 2, figsize=(15, 10), sharex=True, sharey=True)
fig.suptitle('Shared Axes Example', fontsize=16, fontweight='bold')

# Plot with shared axes
axes[0, 0].scatter(df_multiplot['x'], df_multiplot['y'], alpha=0.6, c=df_multiplot['value'])
axes[0, 0].set_title('Plot 1')

axes[0, 1].scatter(df_multiplot['x'], df_multiplot['z'], alpha=0.6, c=df_multiplot['value'])
axes[0, 1].set_title('Plot 2')

axes[1, 0].scatter(df_multiplot['y'], df_multiplot['z'], alpha=0.6, c=df_multiplot['value'])
axes[1, 0].set_title('Plot 3')

axes[1, 1].scatter(df_multiplot['x'], df_multiplot['value'], alpha=0.6, c=df_multiplot['category'])
axes[1, 1].set_title('Plot 4')

plt.tight_layout()
plt.show()

```

```

# Dashboard layout
print("\nDashboard Layout:")

fig = plt.figure(figsize=(20, 15))
fig.suptitle('Data Science Dashboard', fontsize=20, fontweight='bold')

# Create dashboard layout
ax1 = plt.subplot2grid((4, 4), (0, 0), colspan=2, rowspan=2) # Main plot
ax2 = plt.subplot2grid((4, 4), (0, 2), colspan=2) # Top right
ax3 = plt.subplot2grid((4, 4), (1, 2), colspan=2) # Middle right
ax4 = plt.subplot2grid((4, 4), (2, 0), colspan=2) # Bottom left
ax5 = plt.subplot2grid((4, 4), (2, 2), colspan=2) # Bottom right
ax6 = plt.subplot2grid((4, 4), (3, 0), colspan=4) # Bottom full width

# Main plot - Scatter with trend
ax1.scatter(df_multiplot['x'], df_multiplot['y'], c=df_multiplot['value'])
ax1.set_title('Main Analysis: X vs Y', fontsize=14, fontweight='bold')
ax1.set_xlabel('X Variable')
ax1.set_ylabel('Y Variable')

# Top right - Distribution
ax2.hist(df_multiplot['x'], bins=30, alpha=0.7, color='skyblue', edgecolor='black')
ax2.set_title('X Distribution', fontsize=12, fontweight='bold')
ax2.set_xlabel('X Value')
ax2.set_ylabel('Frequency')

# Middle right - Box plot
ax3.boxplot([df_multiplot[df_multiplot['category'] == cat]['value'] for cat in categories])
ax3.set_title('Value by Category', fontsize=12, fontweight='bold')
ax3.set_xticklabels(['A', 'B', 'C', 'D'])
ax3.set_ylabel('Value')

# Bottom left - Time series
ax4.plot(df_multiplot['time'], df_multiplot['trend'], linewidth=2, color='red')
ax4.fill_between(df_multiplot['time'], df_multiplot['trend'], alpha=0.3, color='red')
ax4.set_title('Time Series Trend', fontsize=12, fontweight='bold')
ax4.set_xlabel('Time')
ax4.set_ylabel('Trend Value')

# Bottom right - Correlation heatmap
correlation_data = df_multiplot[['x', 'y', 'z', 'value']].corr()
im = ax5.imshow(correlation_data, cmap='coolwarm', aspect='auto')
ax5.set_title('Correlation Matrix', fontsize=12, fontweight='bold')
ax5.set_xticks(range(len(correlation_data.columns)))
ax5.set_yticks(range(len(correlation_data.columns)))
ax5.set_xticklabels(correlation_data.columns)
ax5.set_yticklabels(correlation_data.columns)

# Bottom full width - Summary statistics
ax6.axis('off')
summary_text = f"""
Data Summary:
• Total samples: {len(df_multiplot):,}
• Categories: {df_multiplot['category'].nunique()}
• Groups: {df_multiplot['group'].nunique()}
• X mean: {df_multiplot['x'].mean():.2f}
• Y mean: {df_multiplot['y'].mean():.2f}
• Value mean: {df_multiplot['value'].mean():.2f}
• Correlation (X,Y): {df_multiplot['x'].corr(df_multiplot['y']):.3f}
"""

```

```
ax6.text(0.1, 0.5, summary_text, fontsize=12, verticalalignment='center',
        bbox=dict(boxstyle='round', facecolor='lightgray', alpha=0.8))

plt.tight_layout()
plt.show()

print("\n=== END OF SUBPLOTS AND MULTIPLE PLOTS ===")
```

11. Saving and Exporting Plots

Production-Ready Output

Creating publication-quality plots requires proper saving and export techniques:

File Formats

- **PNG:** Raster format for web and presentations
- **PDF:** Vector format for publications and printing
- **SVG:** Scalable vector format for web and editing
- **EPS:** Encapsulated PostScript for publications
- **JPG:** Compressed raster format for photos

Resolution and Quality

- **DPI settings:** High resolution for print (300+ DPI)
- **Figure size:** Appropriate dimensions for target medium
- **Bbox settings:** Tight bounding box to remove whitespace
- **Transparency:** Alpha channel support for overlays

Export Best Practices

- **Consistent styling:** Use style sheets for uniform appearance
- **Color management:** Consider colorblind-friendly palettes
- **File naming:** Descriptive names with version control
- **Metadata:** Include plot information in file properties

```
In [ ]: # 11. Saving and Exporting Plots – Production-Ready Output
print("=== SAVING AND EXPORTING PLOTS ===")

# Generate sample data for export examples
np.random.seed(42)
n_samples = 500

export_data = {
    'x': np.random.normal(0, 1, n_samples),
    'y': 2 * np.random.normal(0, 1, n_samples) + 1,
    'category': np.random.choice(['A', 'B', 'C', 'D'], n_samples),
    'value': np.random.exponential(2, n_samples),
    'time': np.linspace(0, 10, n_samples),
    'trend': np.sin(np.linspace(0, 4*np.pi, n_samples)) + 0.1 * np.random
}
```

```

df_export = pd.DataFrame(export_data)

print("Export Dataset Sample:")
print(df_export.head())

# Create a professional plot for export
fig, ax = plt.subplots(figsize=(12, 8))

# Main plot
scatter = ax.scatter(df_export['x'], df_export['y'], c=df_export['value'],
                    cmap='viridis', alpha=0.7, s=50)

# Add trend line
z = np.polyfit(df_export['x'], df_export['y'], 1)
p = np.poly1d(z)
ax.plot(df_export['x'], p(df_export['x']), "r--", alpha=0.8, linewidth=2,

# Customize the plot
ax.set_title('Professional Plot for Export', fontsize=16, fontweight='bold')
ax.set_xlabel('X Variable', fontsize=12)
ax.set_ylabel('Y Variable', fontsize=12)
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3)

# Add colorbar
cbar = plt.colorbar(scatter, ax=ax)
cbar.set_label('Value', fontsize=12)

# Add text annotation
ax.text(0.02, 0.98, f'Sample Size: {len(df_export):,}\nCorrelation: {df_e
        transform=ax.transAxes, fontsize=10, verticalalignment='top',
        bbox=dict(boxstyle='round', facecolor='white', alpha=0.8))

plt.tight_layout()

# Save in different formats
print("\nSaving plots in different formats:")

# 1. PNG - High resolution for print
plt.savefig('professional_plot.png', dpi=300, bbox_inches='tight',
            facecolor='white', edgecolor='none')
print("✓ Saved as PNG (300 DPI)")

# 2. PDF - Vector format for publications
plt.savefig('professional_plot.pdf', bbox_inches='tight',
            facecolor='white', edgecolor='none')
print("✓ Saved as PDF (vector format)")

# 3. SVG - Scalable vector for web
plt.savefig('professional_plot.svg', bbox_inches='tight',
            facecolor='white', edgecolor='none')
print("✓ Saved as SVG (scalable vector)")

# 4. EPS - Encapsulated PostScript
plt.savefig('professional_plot.eps', bbox_inches='tight',
            facecolor='white', edgecolor='none')
print("✓ Saved as EPS (PostScript)")

# 5. JPG - Compressed format
plt.savefig('professional_plot.jpg', dpi=300, bbox_inches='tight',

```

```

        facecolor='white', edgecolor='none', quality=95)
print("✓ Saved as JPG (compressed)")

plt.show()

# Multiple plots for export
print("\nCreating multiple plots for export:")

# Create a multi-panel figure
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle('Multi-Panel Figure for Export', fontsize=18, fontweight='bo

# Plot 1: Scatter plot
axes[0, 0].scatter(df_export['x'], df_export['y'], c=df_export['value'],
                  cmap='viridis', alpha=0.7)
axes[0, 0].set_title('Scatter Plot')
axes[0, 0].set_xlabel('X')
axes[0, 0].set_ylabel('Y')

# Plot 2: Histogram
axes[0, 1].hist(df_export['x'], bins=30, alpha=0.7, color='skyblue', edge
axes[0, 1].set_title('X Distribution')
axes[0, 1].set_xlabel('X Value')
axes[0, 1].set_ylabel('Frequency')

# Plot 3: Box plot
box_data = [df_export[df_export['category'] == cat]['value'] for cat in [
axes[1, 0].boxplot(box_data, labels=['A', 'B', 'C', 'D'])
axes[1, 0].set_title('Value by Category')
axes[1, 0].set_xlabel('Category')
axes[1, 0].set_ylabel('Value')

# Plot 4: Time series
axes[1, 1].plot(df_export['time'], df_export['trend'], linewidth=2, color
axes[1, 1].fill_between(df_export['time'], df_export['trend'], alpha=0.3,
axes[1, 1].set_title('Time Series Trend')
axes[1, 1].set_xlabel('Time')
axes[1, 1].set_ylabel('Trend Value')

plt.tight_layout()

# Save multi-panel figure
plt.savefig('multi_panel_plot.png', dpi=300, bbox_inches='tight',
          facecolor='white', edgecolor='none')
plt.savefig('multi_panel_plot.pdf', bbox_inches='tight',
          facecolor='white', edgecolor='none')
print("✓ Saved multi-panel figure as PNG and PDF")

plt.show()

# Export with custom styling
print("\nExport with custom styling:")

# Set custom style
plt.style.use('seaborn-v0_8')
sns.set_palette("husl")

# Create styled plot
fig, ax = plt.subplots(figsize=(10, 6))

```

```

# Create a more sophisticated plot
x = np.linspace(0, 2*np.pi, 100)
y1 = np.sin(x)
y2 = np.cos(x)
y3 = np.sin(x + np.pi/4)

ax.plot(x, y1, linewidth=3, label='sin(x)', alpha=0.8)
ax.plot(x, y2, linewidth=3, label='cos(x)', alpha=0.8)
ax.plot(x, y3, linewidth=3, label='sin(x +  $\pi/4$ )', alpha=0.8)

# Fill between curves
ax.fill_between(x, y1, alpha=0.2, color='blue')
ax.fill_between(x, y2, alpha=0.2, color='red')

# Customize
ax.set_title('Styled Plot for Export', fontsize=16, fontweight='bold')
ax.set_xlabel('Angle (radians)', fontsize=12)
ax.set_ylabel('Value', fontsize=12)
ax.legend(fontsize=12)
ax.grid(True, alpha=0.3)

# Add annotations
ax.annotate('Peak', xy=(np.pi/2, 1), xytext=(np.pi/2 + 0.5, 1.2),
           arrowprops=dict(arrowstyle='->', color='black', lw=2),
           fontsize=10, ha='center')

plt.tight_layout()

# Save with custom styling
plt.savefig('styled_plot.png', dpi=300, bbox_inches='tight',
           facecolor='white', edgecolor='none')
plt.savefig('styled_plot.pdf', bbox_inches='tight',
           facecolor='white', edgecolor='none')
print("✓ Saved styled plot as PNG and PDF")

plt.show()

# Export with transparency
print("\nExport with transparency:")

# Create plot with transparency
fig, ax = plt.subplots(figsize=(10, 6))

# Generate data
x = np.linspace(0, 10, 100)
y1 = np.sin(x) * np.exp(-x/10)
y2 = np.cos(x) * np.exp(-x/10)

# Plot with transparency
ax.plot(x, y1, linewidth=3, label='Damped Sine', alpha=0.8, color='blue')
ax.plot(x, y2, linewidth=3, label='Damped Cosine', alpha=0.8, color='red')

# Fill with transparency
ax.fill_between(x, y1, alpha=0.3, color='blue')
ax.fill_between(x, y2, alpha=0.3, color='red')

ax.set_title('Plot with Transparency', fontsize=16, fontweight='bold')
ax.set_xlabel('Time', fontsize=12)
ax.set_ylabel('Amplitude', fontsize=12)
ax.legend(fontsize=12)

```



```

ax.grid(True, alpha=0.3)

plt.tight_layout()

# Save with transparency support
plt.savefig('transparent_plot.png', dpi=300, bbox_inches='tight',
            facecolor='none', edgecolor='none', transparent=True)
print("✓ Saved transparent plot as PNG")

plt.show()

# Batch export function
print("\nBatch export function:")

def export_plot(fig, filename_base, formats=['png', 'pdf', 'svg'], dpi=300
    """
    Export a figure in multiple formats.

    Parameters:
    -----
    fig : matplotlib.figure.Figure
        The figure to export
    filename_base : str
        Base filename (without extension)
    formats : list
        List of formats to export
    dpi : int
        Resolution for raster formats
    """
    for fmt in formats:
        if fmt in ['png', 'jpg']:
            fig.savefig(f'{filename_base}.{fmt}', dpi=dpi, bbox_inches='t
                        facecolor='white', edgecolor='none')
        else:
            fig.savefig(f'{filename_base}.{fmt}', bbox_inches='tight',
                        facecolor='white', edgecolor='none')
        print(f"✓ Exported {filename_base}.{fmt}")

# Create a final comprehensive plot
fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle('Comprehensive Data Visualization', fontsize=18, fontweight=

# Plot 1: Scatter with regression
axes[0, 0].scatter(df_export['x'], df_export['y'], c=df_export['value'],
                  cmap='viridis', alpha=0.7)
z = np.polyfit(df_export['x'], df_export['y'], 1)
p = np.poly1d(z)
axes[0, 0].plot(df_export['x'], p(df_export['x']), "r--", alpha=0.8, line
axes[0, 0].set_title('Scatter with Regression')
axes[0, 0].set_xlabel('X')
axes[0, 0].set_ylabel('Y')

# Plot 2: Distribution
axes[0, 1].hist(df_export['x'], bins=30, alpha=0.7, color='skyblue', edge
axes[0, 1].set_title('X Distribution')
axes[0, 1].set_xlabel('X Value')
axes[0, 1].set_ylabel('Frequency')

# Plot 3: Categorical comparison
box_data = [df_export[df_export['category'] == cat]['value'] for cat in [

```

```

axes[1, 0].boxplot(box_data, labels=['A', 'B', 'C', 'D'])
axes[1, 0].set_title('Value by Category')
axes[1, 0].set_xlabel('Category')
axes[1, 0].set_ylabel('Value')

# Plot 4: Time series
axes[1, 1].plot(df_export['time'], df_export['trend'], linewidth=2, color
axes[1, 1].fill_between(df_export['time'], df_export['trend'], alpha=0.3,
axes[1, 1].set_title('Time Series Trend')
axes[1, 1].set_xlabel('Time')
axes[1, 1].set_ylabel('Trend Value')

plt.tight_layout()

# Export in multiple formats
export_plot(fig, 'comprehensive_plot', ['png', 'pdf', 'svg', 'eps'])

plt.show()

# File size comparison
print("\nFile size comparison:")
import os

files = ['professional_plot.png', 'professional_plot.pdf', 'professional_
        'professional_plot.eps', 'professional_plot.jpg']

for file in files:
    if os.path.exists(file):
        size = os.path.getsize(file)
        print(f"{file}: {size:,} bytes ({size/1024:.1f} KB)")

print("\n=== END OF SAVING AND EXPORTING PLOTS ===")
print("\n=== END OF MODULE 7: PLOTTING AND VISUALIZATION ===")
print("\nCongratulations! You have completed Module 7 on Plotting and Vis
print("You now have comprehensive knowledge of:")
print("✓ Matplotlib fundamentals and customization")
print("✓ Seaborn statistical visualization")
print("✓ Multiple plot types and layouts")
print("✓ Professional styling and export techniques")
print("✓ Production-ready visualization skills")

```